

Introduction to Machine Learning

Lecture 19: Sequence Data

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Recap: Sequence Data



"This is the first lecture on ..."



Classic/Jazz/Pop/Hip-hop

"This product is useless!"

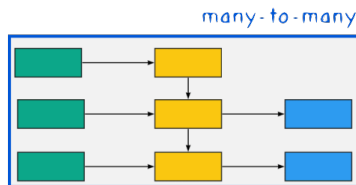
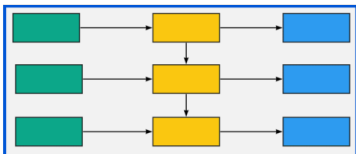
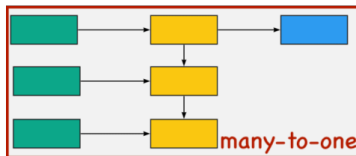
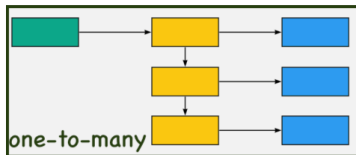


A cute puppy in snow



Sport/Drama/Documentary

Recap: Types of Sequence Problems



Today's Agenda: CNNs and Sequence Data

We continue our final journey which explores briefly

Advanced Topics in Machine Learning

In today's episode, we talk about

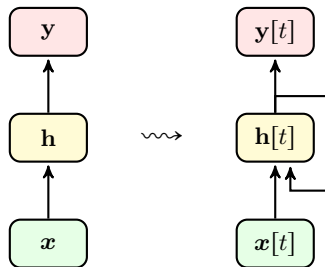
- *Basic Sequence Models*
- *Data Generation*

Recurrent Neural Networks

We can feed a sequence to a neural network, if we include **recurrence**

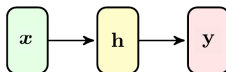
Recurrence

NN takes a state as input and returns the next state as output



Example: *Turning Shallow FNN to Basic RNN*

Say we have a shallow fully-connected FNN



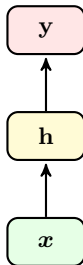
- *It computes hidden feature from the input x*

$$\mathbf{h} = f(\mathbf{W}_1 \mathbf{x})$$

- *It computes the output from the hidden features*

$$\mathbf{y} = f(\mathbf{W}_2 \mathbf{h})$$

Example: *Turning Shallow FNN to Basic RNN*



The model in this case gives us

$$\mathbf{y} \propto P(v|\mathbf{x})$$

and when we train it, we maximize its likelihood

Example: Turning Shallow FNN to Basic RNN

We can make an RNN by using the previous feature in each time: at time t

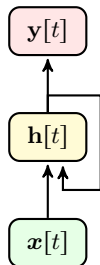
- compute new features from the input $x[t]$ and previous features $\mathbf{h}[t - 1]$*

$$\mathbf{h}[t] = f(\mathbf{W}_1 \mathbf{x}[t] + \mathbf{W}_0 \mathbf{h}[t - 1])$$

- computes the output from the new features*

$$\mathbf{y}[t] = f(\mathbf{W}_2 \mathbf{h}[t])$$

Example: *Turning Shallow FNN to Basic RNN*



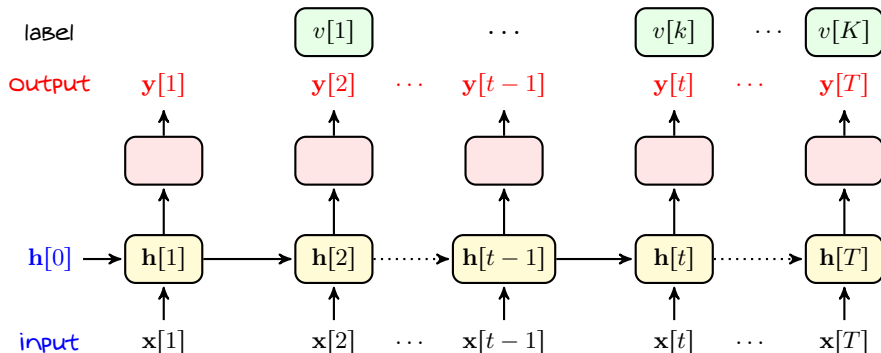
This recursive model determines for us

$$y[t] \propto P(v[t] | \mathbf{h}[t-1], \mathbf{x}[t]) \equiv P(v[t] | \mathbf{x}[1], \dots, \mathbf{x}[t])$$

and we should maximize the likelihood over time

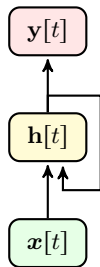
↳ *information about $x[1], \dots, x[t-1]$ is somehow encoded in $\mathbf{h}[t-1]$*

Example: Turning Shallow FNN to Basic RNN



Generic RNN

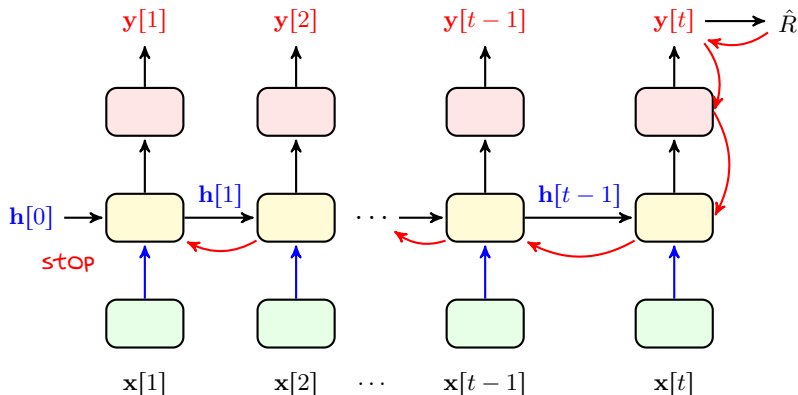
We can do the same thing with any FNN, including CNNs



$h[t]$ is the output of a ny number of hidden layers

Training RNNs

We can apply the same risk minimization; this time over time



To train RNNs, we need to backpropagate through time

Main Challenge: *Vanishing Gradient Through Time*

With long sequences

*backpropagation through time $\approx$ backpropagation through **deep** NNs*

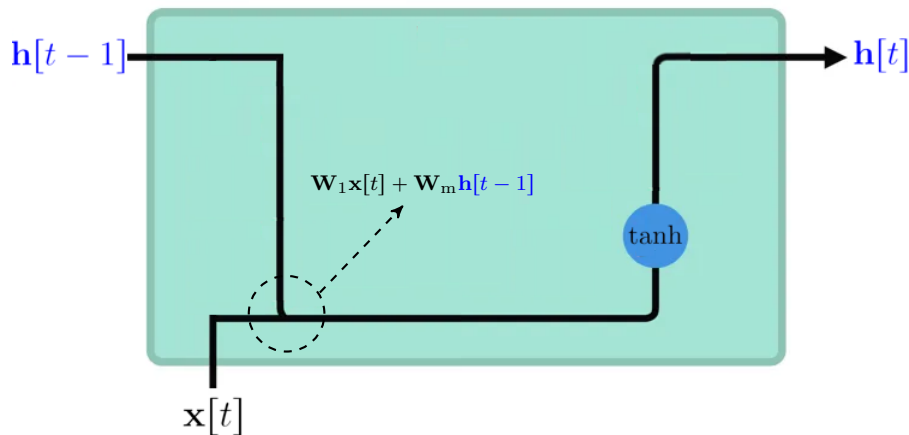
- *Exploding Gradients*
- *Vanishing Gradients*
 - ↳ *Gradients become small over time*
 - ↳ *Weights of RNN are updated only with most recent time instances*
 - ↳ *RNN has a limited memory!*

Classical Remedies

There are various approaches to handle these issues

- *Clipping gradients when exploding*
 - ↳ *Usually used to deal with exploding gradient*
- *Truncated backpropagation through time*
 - ↳ *Updated multiple times in the middle to carry memory forward*
- *Using gated units*
 - ↳ *Use the so-called gate to control better the memory*

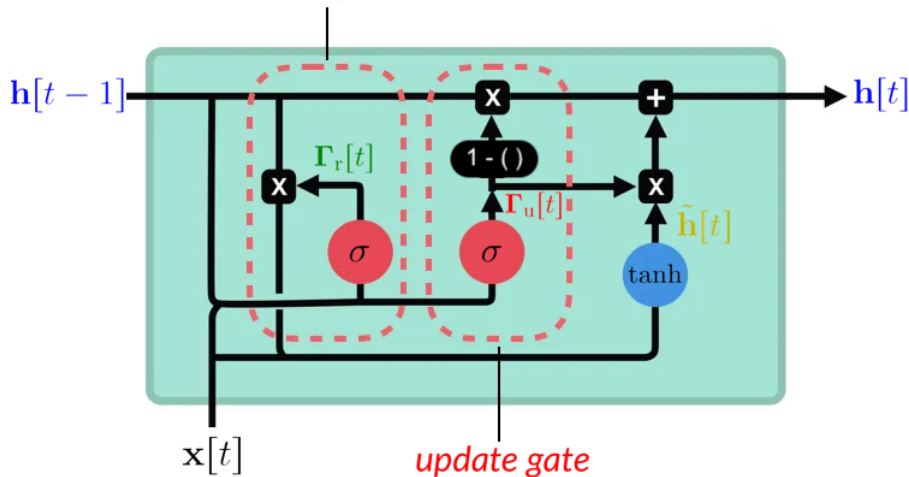
Basic RNN as a Unit



Gated Recurrent Unit: GRU

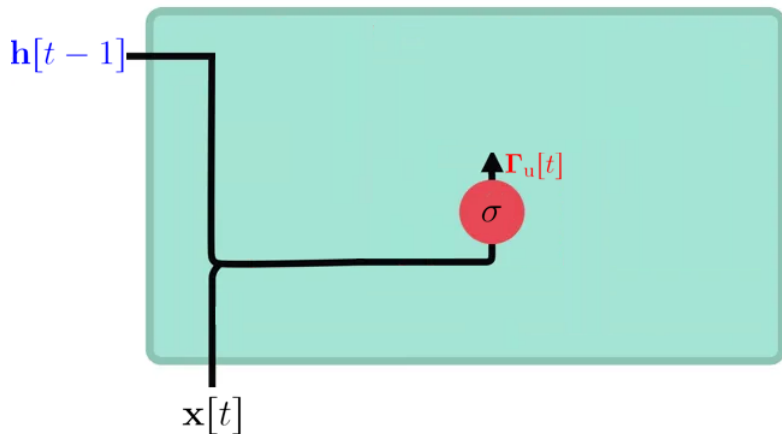
This is what's going on in a GRU cell

reset gate



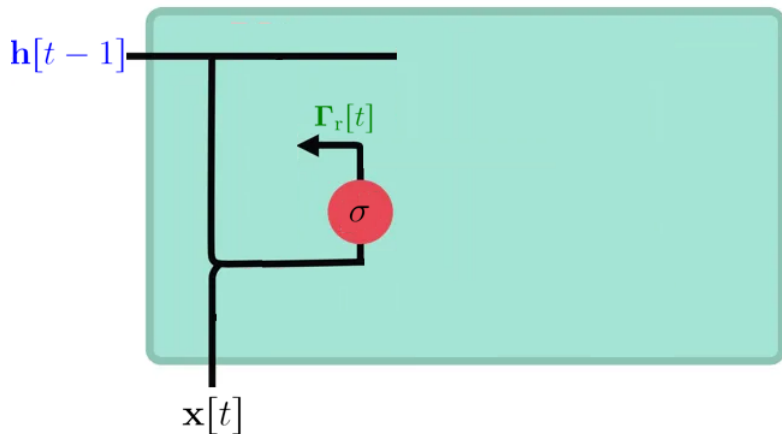
Gated Recurrent Unit: GRU

Compute *update gate* $\Gamma_u[t] = \sigma(\mathbf{W}_{u,\text{in}}\mathbf{x}[t] + \mathbf{W}_{u,\text{m}}\mathbf{h}[t-1])$



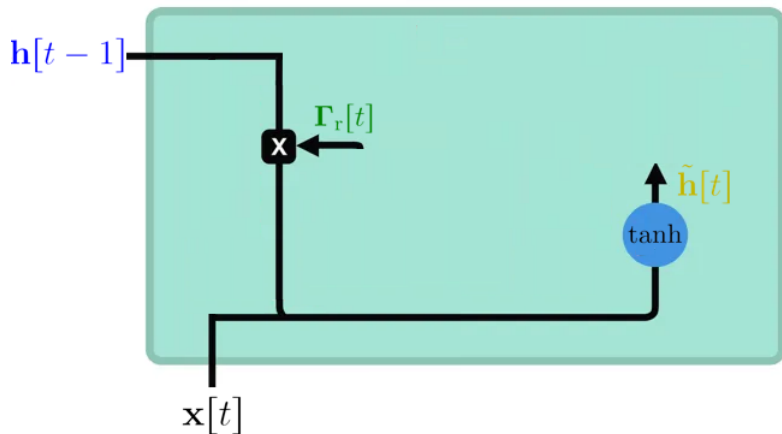
Gated Recurrent Unit: GRU

Compute *reset gate* $\Gamma_r[t] = \sigma(\mathbf{W}_{r,\text{in}}\mathbf{x}[t] + \mathbf{W}_{r,\text{m}}\mathbf{h}[t-1])$



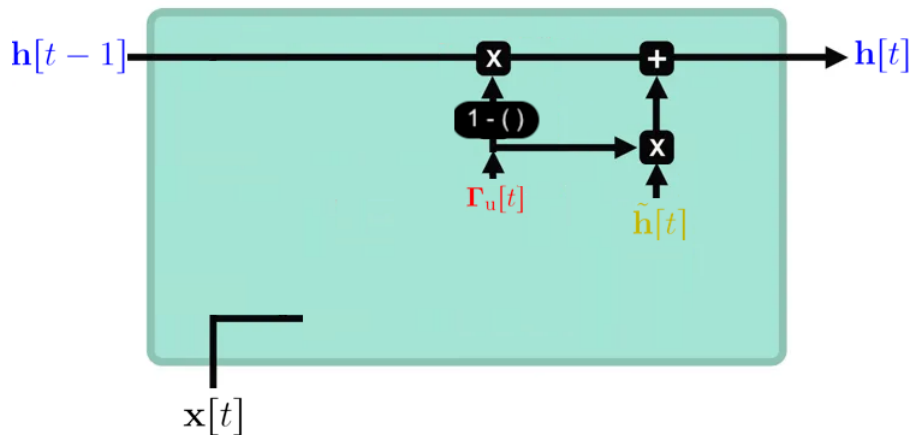
Gated Recurrent Unit: GRU

Compute *actual memory* $\tilde{\mathbf{h}}[t] = f(\mathbf{W}_1 \mathbf{x}[t] + \mathbf{W}_m \mathbf{\Gamma}_r[t] \odot \mathbf{h}[t - 1])$



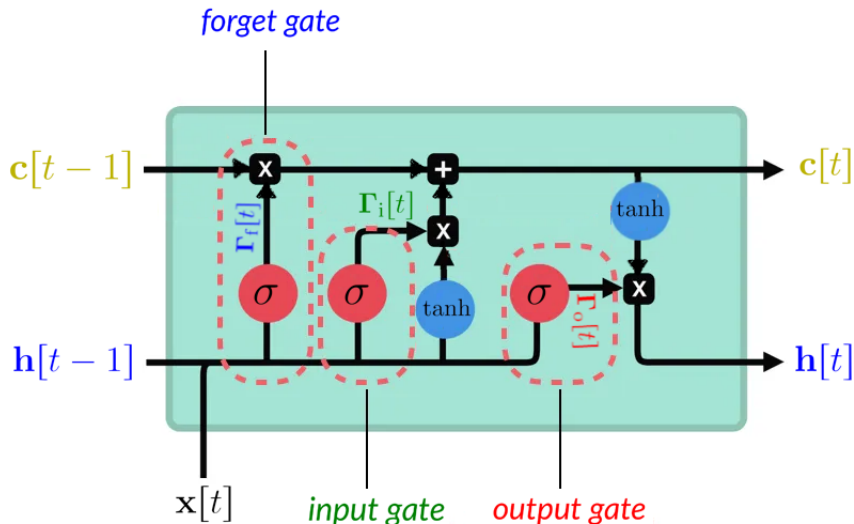
Gated Recurrent Unit: GRU

Update *hidden state* as $\mathbf{h}[t] = (1 - \mathbf{\Gamma}_u[t]) \odot \mathbf{h}[t-1] + \mathbf{\Gamma}_u[t] \odot \tilde{\mathbf{h}}[t]$



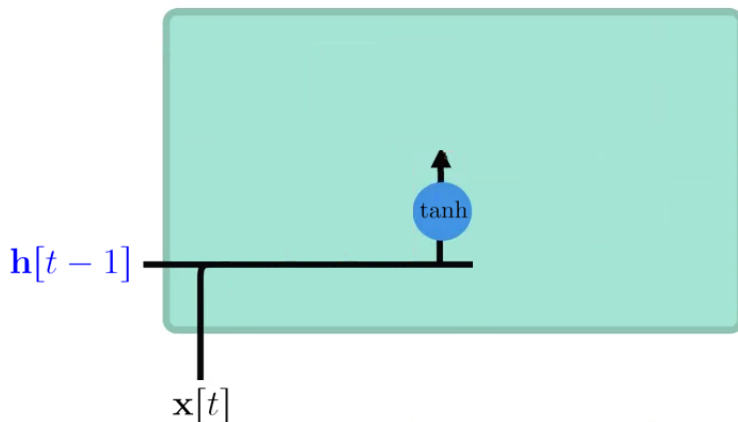
Long Short-Term Memory: *LSTM*

This is how inside an *LSTM unit* looks like



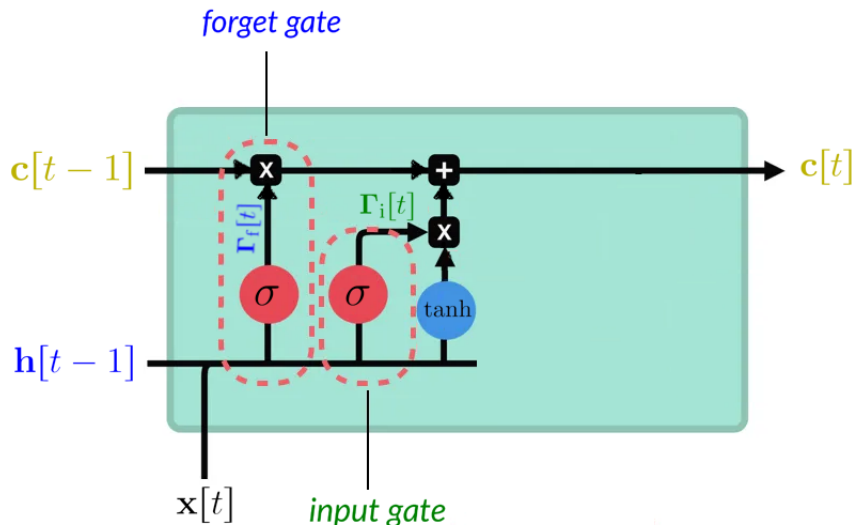
Long Short-Term Memory: *LSTM*

Actual cell state $\tilde{\mathbf{c}}[t] = f(\mathbf{W}_1 \mathbf{x}[t] + \mathbf{W}_m \mathbf{h}[t-1])$



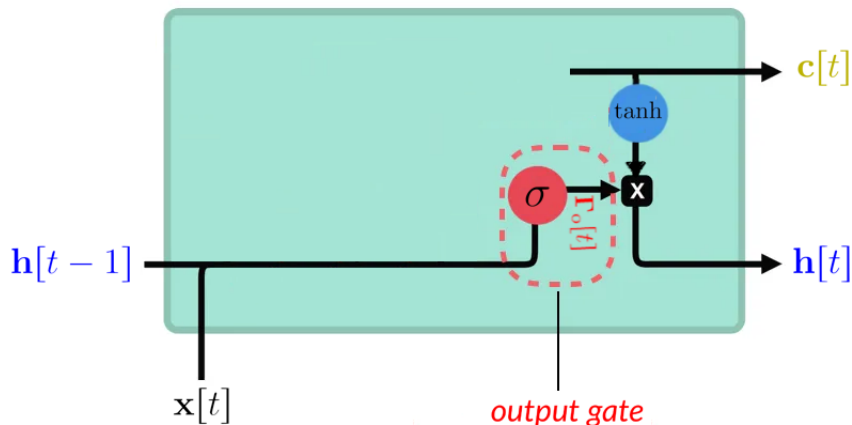
Long Short-Term Memory: *LSTM*

We use *forget gate* and *input gate* to update cell state



Long Short-Term Memory: *LSTM*

We use *output gate* to control fellow of memory to the *hidden state*



Gating is Helpful

- *Using gating we can control vanishing and exploding gradient*
- *LSTM used to be a robust sequence-based model*
 - ↳ *We can still use it for simple tasks like price prediction*
 - ↳ *It was one of the first models used for text generation*
- *At the end of the day, RNNs will still carry limited memory*
 - ↳ *This is why **Attention Mechanism** is developed*
 - ↳ *We can do whole sequence processing based on **Attention***
 - ↳ *This is the idea of **Transformers***

Further Read

- Goodfellow
 - ↳ Chapter 10

RNNs

RNNs and Transformers are discussed in

- *ECE1508: Applied Deep Learning*
 - ↳ *Given in both Fall and Winter Semesters*
- *ECE1786: Creative Applications of NLP*
 - ↳ *Given in Fall Semesters*

First Try: Generate Coherent Text

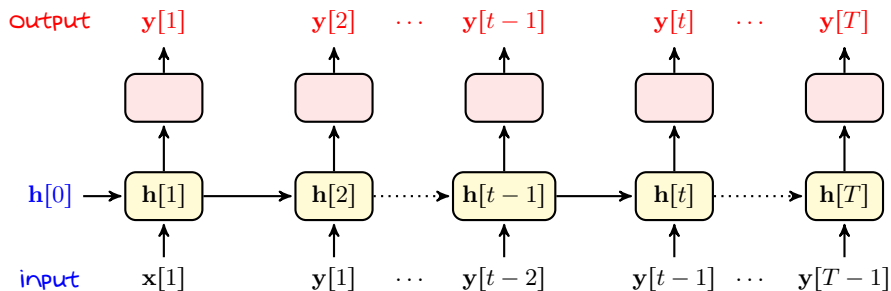
? *Can we make a model generate coherent sentences?*

! *We can use the idea of sequence learning!*

We can construct a supervised learning task

- *Dataset contains many coherent sentences*
 - ↳ *Each sentence is a sequence with each word being $x[t]$*
 - ↳ *Label of each word is the next word*
- *Model is a sequence-based model, e.g., RNN*
- *We minimize the risk computed by cross-entropy*

First Try: Generate Coherent Text



Language Model

Language Model

A model that can output a sequence from natural language with or without an input sequence

We can do lots of things with Language Models

- *Text generation*
- *Auto-caption algorithm*
- *Translators*
- *Reasoning algorithms*

NLP

Language models are key components of natural language processing (NLP)

Alternative Look: *Sampling From Data Distribution*

We can look at text generation alternatively: *an ideal language model in each time t samples from the conditional distribution*

$$P(x[t+1]|x[1:t]) = \text{probability of next word given previous ones}$$

Example: $x[1:t] \equiv$ He does not see what she ...

$$P(x[t+1]|x[1:t]) = \begin{cases} 0 & x[t+1] = \text{see} \\ 0 & x[t+1] = \text{am} \\ \neq 0 & x[t+1] = \text{sees} \\ \neq 0 & x[t+1] = \text{can} \\ \vdots & \end{cases}$$

Ideally, language model chooses right words as frequent as probability says

Language Model: *Alternative Look*

In practice, we try to **learn** this distribution using a model

Language Model

Language models learn $P(x[t+1]|x[1:t])$ using a learning model

$$f_{\theta}(x[t+1]|x[1:t])$$

where f_{θ} is realized via a hypothesis, e.g., is set to an RNN

Example: *The output of RNN computes a Softmax*

- *It computes the probability of next word*
- *If model gets trained properly, it can compute right probability*
- *We are sampling from this distribution when we infer the model*

More Abstract Viewpoint

? *Can we generate something else, as well?*

! *Sure! We only need to sample correctly*

Say we want to generate a cat image of size $3 \times 224 \times 224$: we would say that this is a random object $\mathbf{X}$ from distribution $P_{data}(\mathbf{X})$

Data Distribution

$P_{data}(\mathbf{X})$ is the distribution of all images $\mathbf{X}$ that are cat, i.e.,

$$P_{data}(\mathbf{X} = \text{DOG}) = 0$$

If we know $P_{data}(\mathbf{X})$ and are able to sample from it, we could

generate a sample cat image by drawing a sample from $P_{data}(\mathbf{X})$

More Abstract Viewpoint

? *Can we find $P_{data}(\mathbf{X})$?*

! *No! But, we can learn it from its samples!*

Any cat image $\mathbf{X}_n$ is a sample from $P_{data}(\mathbf{X})$: we put them in dataset

$$\mathbb{D} = \{\mathbf{X}_1, \dots, \mathbf{X}_N\}$$

We then try learning $P_{data}(\mathbf{X})$ using a model $f_{\theta}(\mathbf{X})$

- *We compute the likelihood of dataset*
- *We find θ that $f_{\theta}(\mathbf{X})$ is close to $P_{data}(\mathbf{X})$*

Approach I: Learning Distribution Directly

? But we don't know $P_{data}(\mathbf{X})$?!

! We can do it without knowing $P_{data}(\mathbf{X})$ explicitly

Using $\mathbb{D}$, we can compute the likelihood of θ

$$\mathcal{L}_{\mathbb{D}}(\theta) = \prod_n f_{\theta}(\mathbf{X}_n)$$

Maximum Likelihood Estimation

Find θ such that the likelihood is maximized

$$\theta^{\star} = \underset{\theta}{\operatorname{argmax}} \log \mathcal{L}_{\mathbb{D}}(\theta)$$

Approach I: Learning Distribution Directly

If we have **enough** samples and **complex** model; then, we see that

$$f_{\theta}(\mathbf{X}) \approx P_{data}(\mathbf{X})$$

However, we should note that

- **Enough** is too much and **complex** is too complicated
- Even if we get a good approximation, we still need to sample it
 - ↳ We may use Monte-Carlo or Gibbs sampling
 - ↳ They can yet be slow or imprecise

This approach was one of first approaches for learning $P_{data}(\mathbf{X})$ that is used in

- ↳ Flow-based models, energy-based models, . . .

Approach II: Sampling From Data Distribution

People gradually started to think

? Why do not we learn to *directly sample* from $P_{data}(\mathbf{X})$?!

Say we have a noise process like $\mathbf{Z}$: if we pass this random process it through a function $f(\mathbf{Z})$; then, we get an output $\mathbf{X} = f(\mathbf{Z})$

for known noise distribution, the distribution of $\mathbf{X}$ depends on $f(\cdot)$

If we design $f(\cdot)$ properly, we can make $\mathbf{X}$ to be distributed with $P_{data}(\mathbf{X})$!

Approach II: Sampling From Data Distribution

Learn to Sample

Starting from a noise process $\mathbf{Z}$, we approximate a function $f(\cdot)$ by a model $f_{\theta}(\cdot)$, such that $\mathbf{X} = f_{\theta}(\mathbf{Z})$ is approximately distributed with $P_{data}(\mathbf{X})$

This approach simplifies the process significantly: to generate a sample $\mathbf{X}$

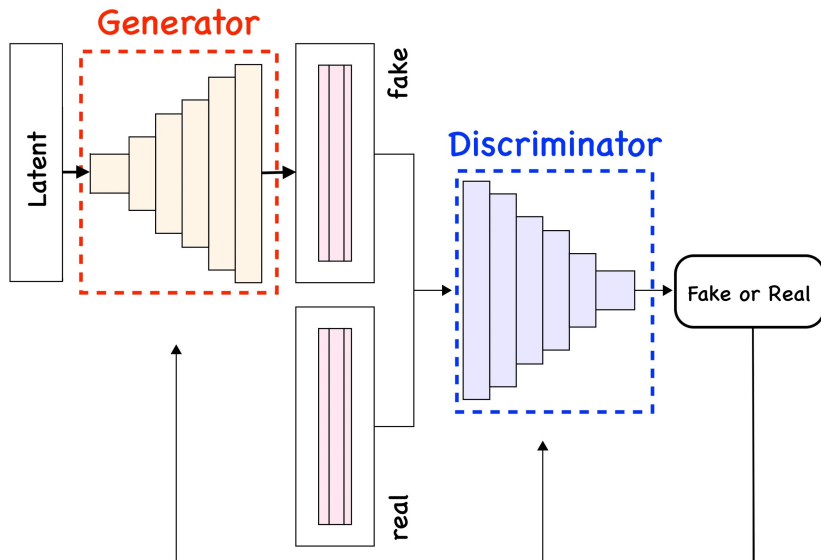
- We sample noise $\mathbf{Z}$
 - ↳ This is easy! For instance, we sample a Gaussian distribution!
- We pass the noise sample through the trained model as $\mathbf{X} = f_{\theta}(\mathbf{Z})$ and get the data sample

Approach II: *Sampling From Data Distribution*

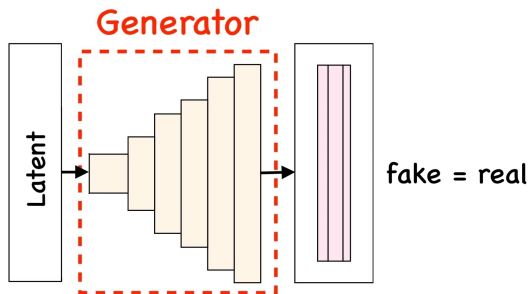
There are three well-known approaches to learn sampling

- *Generative Adversarial Networks (GANs)*
 - ↳ *They learn to sample by a min-max game*
- *Variational Auto-Encoders (VAEs)*
 - ↳ *They use auto-encoding to learn sampling*
- *Diffusion Models*
 - ↳ *They learn sampling through diffusion process*

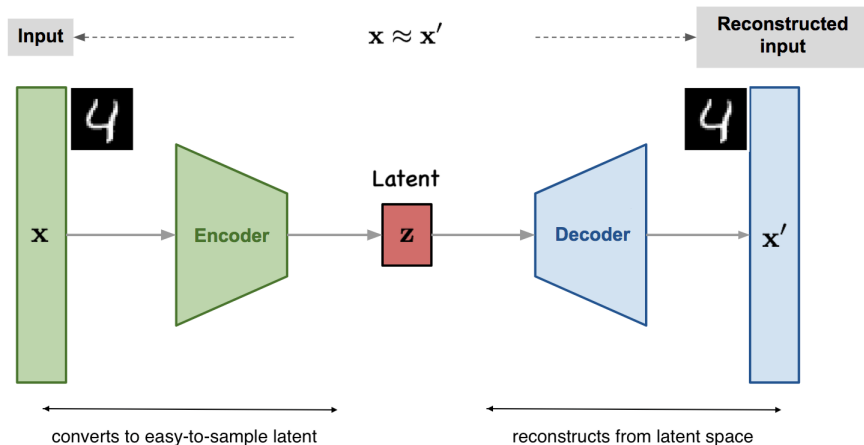
Generative Adversarial Networks (GANs): *Training*



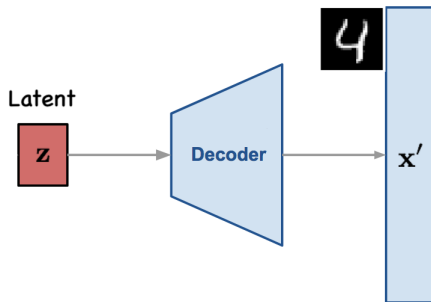
Generative Adversarial Networks (GANs): *Generation*



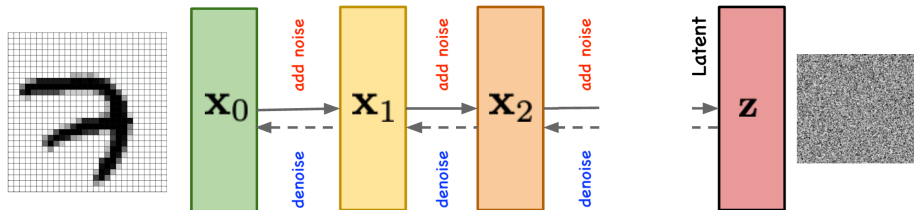
Variational Auto-Encoders (VAEs): Training



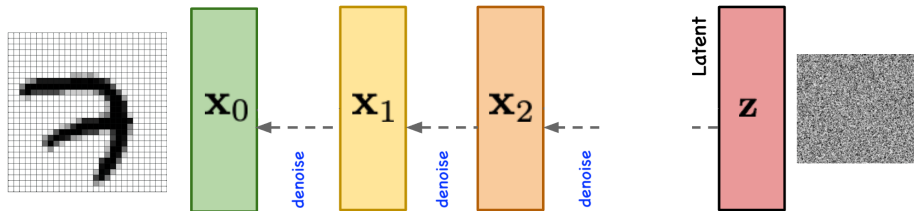
Variational Auto-Encoders (VAEs): *Generation*



Diffusion Models: *Training*



Diffusion Models: *Generation*



Generative vs Discriminative Models

What we used throughout the course for classification were *discriminative models* which compute $P(v|\mathbf{x})$. We could use *generative models* as well to classify: we learn $P(\mathbf{x}, v)$ from samples in dataset and then write

$$P(v|\mathbf{x}) = \frac{P(\mathbf{x}, v)}{P(\mathbf{x})}$$

We however look for v with highest $P(v|\mathbf{x})$, so we have

$$\operatorname{argmax}_v P(v|\mathbf{x}) = \operatorname{argmax}_v P(\mathbf{x}, v)$$

So, we don't need to compute $P(\mathbf{x})$ anymore and can directly use $P(\mathbf{x}, v)$!

- The simplest classifier of this form is the *naive Bayes classifier*
- For basic classification tasks, we still prefer *discriminative models*

Further Read

- Goodfellow
 - ↳ Chapter 20

Generative Models